

1. Analyze the two GenAI programs. The analysis must include:

a. Name of two GenAI programs

Claude

ChatGPT

b. How you accessed them (e.g., Google query, CoPilot on my computer)

Both through google so the website for both of the AI's

c. Prompt used. (You must use the same prompt for both GenAIs.)

I used two - 3 images of the assignment requirements on the second page and then the test file format and the sample text.

d. Generated code from both GenAIs.

I cant paste both here because they are both about 500 lines of code and comments, but they are in the repo and easy to access if you have this pdf

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the "GenAI Code Comparison" lecture.

Claude

1. Correctness - Outputs match to the expected outputs. The heap is used correctly and implemented correctly by using it with a array and list. While also finding the correct spot for new additions to the heap and adjusting the heap upon removals of emails. Edge cases are also handled and addressed.
2. Execution Time - The time is based upon the amount of emails and commands associated with the file. But each of the functions have different notations and when you combine for the worst case its $O((n + m) \log n)$. EMAIL (insert) is $O(\log n)$, NEXT is $O(1)$, READ is $O(\log n)$, COUNT $O(1)$.
3. Space Complexity - Storage for emails is $O(n)$ for n amount of emails. The array grows by doubling using the `resizeHeapIfNeeded` which isn't amazing but is just being cautious in case expanding the heap is needed.
4. Maintainability - It has good overall maintainability but it may be a little more difficult for people to understand and lower level programmers will struggle to understand a lot of the code with or without comments so thats a fairly large negative. Its good that every function serves and purpose and not rarely used they are all used and needed so theres no need to find a functions reason to be in the program.

ChatGPT

1. Correctness - The code also implements a max heap correctly with a array without using a pre-existing library. Each email has its information tied to it with a number thats just the date

reversed. The priority function assigns values to the words in the email that need to come before the others below it. One issue is the max size of 1000 emails for the heap. All commands in the emails are also handled well and don't cause errors or crashes when running the program.

2. Execution Time - This code has good execution time and better than Claude's with a $O(\log n)$ time. NEXT and COUNT are $O(1)$ time and READ is the worst case of $O(\log n)$.
3. Space Complexity - Overall space complexity is also $O(n)$ for n amount of emails. The heap only allows up to 1000 emails so the hypothetical max is $O(1000)$.
4. Maintainability - This code has a better maintainability than Claude's in my opinion because of the much simpler methods used and straightforward thinking when it comes to approaches on each of the functions. Many ways the heap is altered are simple concepts and typical with how they are used. It has extensive comments on almost every line, and more descriptive comments on lines that are confusing or harder to interpret. The only issue is similar to Claude's with the size being stuck at 1000 because you'd have to remove the limit and then address and adjust the rest of the code in the file.

f. Select one to use as the basis for this assignment and justify why you picked it.

I would use the ChatGPT code because I personally like that it's much simpler and compact. It keeps it simple while Claude gets a little complicated for something that doesn't need to be super complex.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

1. Correctness - Added validation for the sender categories and added checks for memory allocation failures. Rejects bad emails rather than just inserting them into the heap.
2. Execution Time - Couldn't find a way to further improve the execution time better than $O(\log n)$ so it's remaining the same.
3. Space Complexity - Replaced the limited 1000 limit list with an allocated array. It starts with space for 10 and doubles when needed similar to Claude's system. It fixes the limit of 1000 and having to reserve 1000 spaces for the list. Still remains $O(n)$.
4. Maintainability - Changed memory management into three separate functions, and also changed the comparisons to use pointers, and added the named constants for capacity and max line. Helps with changing specifics and not messing up trying to tweak small things.

Command lines to run programs if needed

ChatGPT

```
gcc -Wall -Wextra -o chatgpt_email_priority chatgpt_email_priority.c
```

```
./chatgpt_email_priority chatgpt_test1.txt
```

```
./chatgpt_email_priority chatgpt_test2.txt
```

Claude

```
gcc -Wall -Wextra -o claude_email_priority claude_email_priority.c
```

```
./claude_email_priority chatgpt_test1.txt
```

```
./claude_email_priority chatgpt_test2.txt
```

Improved ChatGPT

```
gcc -Wall -Wextra -o chatgpt_email_priorityimproved chatgpt_email_priorityimproved.c
```

```
./chatgpt_email_priorityimproved chatgpt_test1.txt
```

```
./chatgpt_email_priorityimproved chatgpt_test2.txt
```